

CH 2.1: DISTRIBUTED DATABASE DESIGN

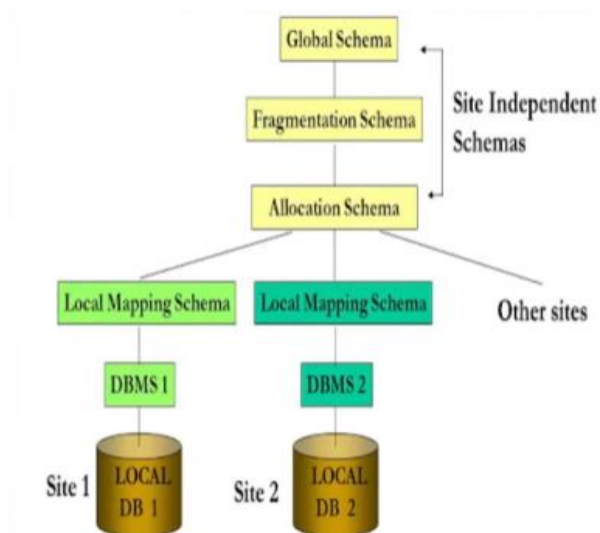
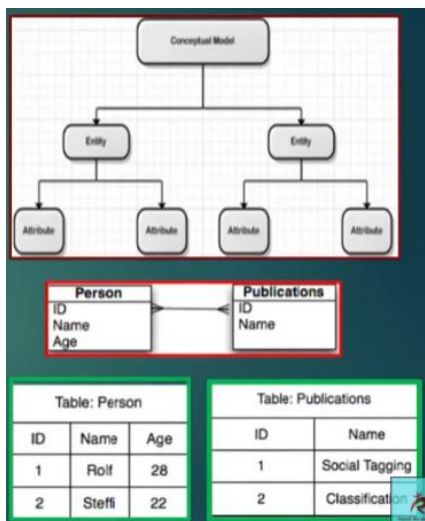
Contents

Contents	1
Alternative Design Approaches	1
Loosely and Tightly Coupled System	2
Fragmentation.....	3
Correctness Rules in Fragmentation	3
Horizontal Fragmentation:	4
Types of Horizontal Fragmentation:	4
1. Primary Fragmentation.....	4
2. Derived Fragmentation.....	4
Min term predicates	5
Vertical Fragmentation.....	6
Mixed (Hybrid) Fragmentation	7
Data Allocation.....	9

Alternative Design Approaches

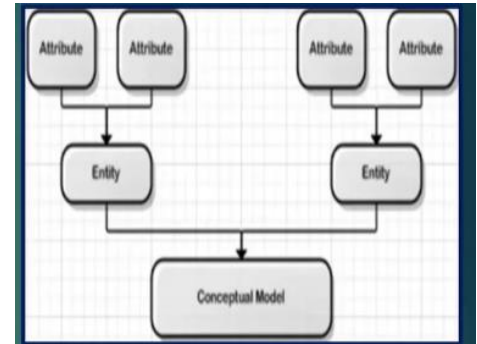
1. Top-Down Approach

- The design starts from a **global schema**.
- First, the **entire system requirements** are analyzed.
- A **global conceptual schema** is designed as if the system is centralized.
- Then, it is **partitioned** and **allocated** to different sites.
- Suitable when the organization has a clear view of global data needs.
- Ensures **data integration and consistency** from the beginning.
- **Example:** Design a company-wide employee database first, then divide it among regional offices.



2. Bottom-Up Approach

- Begins with **existing local databases** at various sites.
- Each site's **schemas are analyzed** and documented.
- Then, these local schemas are **integrated** into a **global schema**.
- Suitable when the system has **autonomous databases** already in use.
- More complex due to **schema mismatches and conflicts**.
- **Example:** Combine separate HR, finance, and sales databases from different branches into one global view.



Loosely and Tightly Coupled System

Loosely Coupled and Tightly coupled System

Distributed Systems
 " A Distributed System is a collection of independent computers that appears to its users as a single coherent system "

12

Tightly Coupled Systems

- Systems with a single system wide memory
- Parallel Processing System , SMMP (shared memory multiprocessor systems)

Loosely Coupled System

- Distributed Memory Systems (DMS)
- Communication via Message Passing

1. Tightly coupled is a concept of system design and computing where every hardware & software components that are linked together in such manner that each component is dependent upon each other.

2. Key point to be noticed is it shares the memory across the CPU's. Communication will be established using shared memory.

1. Every module is somewhat independent of each other

2. In a loosely coupled system, hardware and software may interact but they are not dependent on each other.

3. Loosely Coupled Systems are actually Distributed Systems. Because System which does not share memory with other systems and have their own clock (indicates how fast the CPU can run). It is a Distributed system

[SUBSCRIBE](#)

Feature	Tightly Coupled System	Loosely Coupled System
Definition	Systems with shared memory and close integration	Systems with distributed memory and components
Components	Closely connected	Independent and autonomous
Communication	High-speed via shared memory	Message passing over network
Performance	Faster	Slower compared to tightly coupled
Scalability	Less scalable	Highly scalable
Fault Tolerance	Low – failure affects entire system	High – failure isolated
Example	Multi-core processors	Distributed systems (e.g., cloud)
Maintenance	Harder to modify	Easier to upgrade or replace parts

Fragmentation

- Fragmentation is the process of **dividing a database into smaller pieces** (fragments) that can be **stored at different sites** in a distributed database system.
- Each fragment is stored at one or more sites and is **logically related to the original database**.
- Fragmentation should be carried out in such a way that **reconstruction** of original database from the fragments is possible.
- Fragmentation helps in improving **efficiency, parallelism, and locality of data access**.

Correctness Rules in Fragmentation

When fragmenting a relation in distributed databases, certain **correctness rules** must be satisfied to ensure:

- We should not lose data because of fragmentation
- We should not get redundant data because of fragmentation.

To ensure these properties we need to verify if fragmentation is correctly performed or not. Due to this we have the following correctness rules:

1. Completeness

- Every data item that is in the original relation must appear in at least one fragment.
- No tuple from the original relation should be missing after fragmentation.

$$R_1, R_2, \dots, R_n \in R$$

- **Example:** If the original table EMP has 100 tuples, then the total of all tuples across the fragments must also be 100.

2. Reconstruction (Recomposability)

- It must be possible to reconstruct the original relation by applying a relational operation (usually **UNION** for horizontal fragmentation and **JOIN** for vertical fragmentation) on the fragments.
- The system must be able to retrieve the original relation without any loss or duplication of data.
 - **Horizontal Fragmentation:** $R = R_1 \cup R_2 \cup \dots \cup R_n$
 - **Vertical Fragmentation:** $R = \pi_{a1}(R_1) \bowtie \pi_{a2}(R_2) \bowtie \dots \bowtie \pi_{an}(R_n)$

3. Disjointness (for horizontal fragmentation)

- The data in each fragment must be **mutually exclusive** — no data item should appear in more than one fragment.
- This ensures that each tuple belongs to exactly one fragment, preventing data duplication.
- If a table R is partitioned into fragments $R_1, R_2, \dots, R_n$ then disjointness insists the following:

$$R_1 \cap R_2 \cap \dots \cap R_n = \emptyset$$

- **Example:** If EMP1 contains employees from Delhi, then EMP2 (e.g., Mumbai) should not include any Delhi employee.

Degree of Fragmentation: The degree of fragmentation refers to the extent to which a database relation (table) is divided into fragments. It reflects how many fragments a relation has after applying a fragmentation strategy.

Horizontal Fragmentation:

- Divides a relation (table) **row-wise** based on certain **conditions or predicates**.
- Each fragment contains a subset of **tuples (rows)**.
- Every fragment has all the columns and a subset of origin relation.
- All fragments are stored in different sites and each fragment are distinct tuples.
- We can distribute the pieces to the sites of distributed database however we like and reassemble them.

Consider a table:

```
EMPLOYEE(EID, Name, City, Salary)
```

- Fragment 1: All employees in Delhi

```
EMP_DELHI = SELECT * FROM EMPLOYEE WHERE City = 'Delhi';
```

- Fragment 2: All employees in Mumbai

```
EMP_MUMBAI = SELECT * FROM EMPLOYEE WHERE City = 'Mumbai';
```

Types of Horizontal Fragmentation:

1. **Primary Fragmentation:** Primary fragmentation is the initial and independent division of a global relation (table) based on either horizontal, vertical, or mixed fragmentation.
 - Applied directly on the original global table.
 - Each primary fragment is complete and disjoint.
 - Used to distribute data based on access patterns or location-specific requirements.

Example:

```
CUSTOMER(CustID, Name, City)
```

Horizontal primary fragmentation based on `City` directly derived from the global `CUSTOMER` table:

- `CUST_Mumbai = SELECT * FROM CUSTOMER WHERE City = 'Mumbai'`
- `CUST_Delhi = SELECT * FROM CUSTOMER WHERE City = 'Delhi'`

2. **Derived Fragmentation:** Derived fragmentation occurs when a relation is fragmented based on the fragmentation of another related relation, typically through foreign key relationships.
 - Used when two tables are logically related.
 - Maintains referential integrity.
 - Ensures co-location of related data to reduce join costs.

Suppose we have two relations:

1. `CUSTOMER(CustID, Name, City)`
2. `ORDERS(OrderID, CustID, Product, Amount)`

If `CUSTOMER` is horizontally fragmented by city:

- `CUST_Mumbai`
- `CUST_Delhi`

Then `ORDERS` can be **derived fragmented** using `CustID` from the already fragmented `CUSTOMER` table:

- `ORDERS_Mumbai = SELECT * FROM ORDERS WHERE CustID IN (SELECT CustID FROM CUST_Mumbai)`
- `ORDERS_Delhi = SELECT * FROM ORDERS WHERE CustID IN (SELECT CustID FROM CUST_Delhi)`

This way, each `ORDERS` fragment corresponds to the respective `CUSTOMER` fragment.

Min term predicates

Primary Horizontal Fragmentation (PHF)

Primary Horizontal Fragmentation is about fragmenting a single table horizontally (row wise) using a set of simple predicates (conditions).

25

What is simple predicate?

Given a table R with set of attributes $[A_1, A_2, \dots, A_n]$, a simple predicate P_i can be expressed as follows:

$P_i : A_i \theta \text{ Value}$

Where θ can be any of the symbols in the set $\{=, <, >, \leq, \geq, \neq\}$, value can be any value stored in the table for the attributed A_i .

Acno	Balance	Branch_Name
A101	5000	Mumbai
A103	10000	New Delhi
A104	2000	Chennai
A102	12000	Chennai
A110	6000	Mumbai
A115	6000	Mumbai
A120	2500	New Delhi

Figure 1: Account table

Example 1: for the table Account, if simple conditions are, $\text{Balance} < 10000$, $\text{Balance} \geq 10000$, then, Set of simple predicates $P_1 = \{\text{Balance} < 10000, \text{Balance} \geq 10000\}$

Example 2: if simple conditions are, $\text{Branch_name} = \text{'Chennai'}$, $\text{Branch_name} = \text{'Mumbai'}$, $\text{Balance} < 10000$, $\text{Balance} \geq 10000$, then, Set of simple predicates $P_2 = \{\text{Branch_name} = \text{'Chennai'}$, $\text{Branch_name} = \text{'Mumbai'}$, $\text{Balance} < 10000$, $\text{Balance} \geq 10000\}$

SUBSCRIBE

Min-term Predicate

Min-term predicate which can be written as follows;

Min-term predicate, $M_i = P_1 \wedge P_2 \wedge P_3 \wedge \dots \wedge P_n$

- Here, P_1 means both P_1 or $\neg(P_1)$, P_2 means both P_2 or $\neg(P_2)$, and so on. Using the conjunctive form of various simple predicates in different combination, we can derive many such min-term predicates.
- For **Example 1:** for the table Account, if simple conditions are, $\text{Balance} < 10000$, $\text{Balance} \geq 10000$, then,
 - We can derive set of min-term predicates using the rules stated above as follows;
 - We will get 2^n min-term predicates, where n is the number of simple predicates in the given predicate set. For P_1 , we have 2 simple predicates. Hence, we will get 4 (2^2) possible combinations of min-term predicates as follows;
 - $m_1 = \{\text{Balance} < 10000 \wedge \text{Balance} \geq 10000\}$
 - $m_2 = \{\text{Balance} < 10000 \wedge \neg(\text{Balance} \geq 10000)\}$
 - $m_3 = \{\neg(\text{Balance} < 10000) \wedge \text{Balance} \geq 10000\}$
 - $m_4 = \{\neg(\text{Balance} < 10000) \wedge \neg(\text{Balance} \geq 10000)\}$

Our next step is to choose the min-term predicates which can satisfy certain conditions to fragment a table, and eliminate the others which are not useful.

- $\text{Account1} = \sigma_{\text{Balance} < 10000 \wedge \text{Balance} \geq 10000}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account1} \leftarrow \text{SELECT } * \text{ FROM account WHERE balance} < 10000 \text{ AND balance} \geq 10000;$
- $\text{Account2} = \sigma_{\text{Balance} < 10000 \wedge \neg(\text{Balance} \geq 10000)}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account2} \leftarrow \text{SELECT } * \text{ FROM account WHERE balance} < 10000 \text{ AND NOT balance} \geq 10000;$
where NOT balance ≥ 10000 is equivalent to balance < 10000 .
- $\text{Account3} = \sigma_{\neg(\text{Balance} < 10000) \wedge \text{Balance} \geq 10000}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account3} \leftarrow \text{SELECT } * \text{ FROM account WHERE NOT balance} < 10000 \text{ AND balance} \geq 10000;$
where NOT balance < 10000 is equivalent to balance ≥ 10000 .
- $\text{Account4} = \sigma_{\neg(\text{Balance} < 10000) \wedge \neg(\text{Balance} \geq 10000)}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account4} \leftarrow \text{SELECT } * \text{ FROM account WHERE NOT balance} < 10000 \text{ AND NOT balance} \geq 10000;$

27

■ The first query for fragment Account1 (min-term predicate m1) is invalid as any record in a table cannot have two values for any attribute in one record. That is, the condition $(Balance < 10000 \wedge Balance \geq 10000)$ requires that the value for balance must both be less than 10000 and greater and equal to 10000, which is not possible. Hence the condition violates and can be eliminated.

■ For fragment Account2 (min-term predicate m2), the condition is $(balance < 10000 \text{ and } balance < 10000)$ which ultimately means $balance < 10000$ which is correct.

■ Likewise, fragment Account3 is valid and Account4 must be eliminated.

Account1 = $\sigma_{Balance < 10000 \wedge Balance \geq 10000}(Account)$
 which can be written in equivalent SQL query as,
 Account1 $\leftarrow$ SELECT * FROM account WHERE balance < 10000 AND balance $\geq$ 10000;

Account2 = $\sigma_{Balance < 10000 \wedge \neg(Balance \geq 10000)}(Account)$
 which can be written in equivalent SQL query as,
 Account2 $\leftarrow$ SELECT * FROM account WHERE balance < 10000 AND NOT balance $\geq$ 10000;
 where NOT balance $\geq$ 10000 is equivalent to balance < 10000.

Account3 = $\sigma_{\neg(Balance < 10000) \wedge Balance \geq 10000}(Account)$
 which can be written in equivalent SQL query as,
 Account3 $\leftarrow$ SELECT * FROM account WHERE NOT balance < 10000 AND balance $\geq$ 10000;
 where NOT balance < 10000 is equivalent to balance $\geq$ 10000.

Account4 = $\sigma_{\neg(Balance < 10000) \wedge \neg(Balance \geq 10000)}(Account)$
 which can be written in equivalent SQL query as,
 Account4 $\leftarrow$ SELECT * FROM account WHERE NOT balance < 10000 AND NOT balance $\geq$ 10000;

SELECT * FROM account WHERE balance < 10000;

Account ₂		
Acno	Balance	Branch_Name
A101	5000	Mumbai
A104	2000	Chennai
A120	2500	New Delhi
A110	6000	Mumbai
A115	6000	Mumbai

Account ₃		
Acno	Balance	Branch_Name
A103	10000	New Delhi
A102	12000	Chennai

Vertical Fragmentation

Divides a relation **column-wise**, keeping the **primary key** in all fragments to allow reconstruction via join.

- Fragment 1: EMP_NAMES = PROJECT EID, Name FROM EMPLOYEE;
- Fragment 2: EMP_DETAILS = PROJECT EID, City, Salary FROM EMPLOYEE;

id	Name	Sex	Age
1	Sebastian	M	3
2	John	M	5
3	Garry	M	8
4	Louis	F	7

Original Table

Vertical Fragmentation

id
1
2
3
4

Name
Sebastian
John
Garry
Louis

Sex
M
M
M
F

Age
3
5
8
7

Two approaches in vertical fragmentation:

1. Grouping (Bottom up approach)

- Grouping refers to **combining related attributes** together into fragments.
- To optimize query performance by keeping frequently accessed attributes together.
- Example:**

Consider relation EMP(EID, EName, Dept, Salary, Phone) We can group:

- Fragment 1: EMP1(EID, EName, Dept) — for HR-related queries
- Fragment 2: EMP2(EID, Salary, Phone) — for payroll-related queries

Each fragment must include the primary key (EID) to enable reconstruction via natural join.

2. Splitting (Top down approach)

- Splitting refers to **dividing a relation's attributes into smaller sets** across different fragments.
- To distribute attributes that are used independently or to reduce data transfer.
- Example:**

From the above relation, you may split into:

- Fragment 1: EMP1(EID, EName)
- Fragment 2: EMP2(EID, Dept)
- Fragment 3: EMP3(EID, Salary)
- Fragment 4: EMP4(EID, Phone)

This is a **fine-grained** approach, useful when queries typically access only a subset of attributes.

Example of Vertical Fragmentation:

Eid	Ename	Salary	Bonus	Project	Experience
1	ABC	50,000	5000	P ₁	5
2	DEF	40,000	7000	P ₂	3
3	GHI	45,000	8000	P ₃	2

We will make fragments based on relevant information

- Primary key is included in every fragment.
- **Fragment1** - Basic Employee Details(Eid,Ename,Salary)
- **Fragment2** - Bonus on Projects(Eid,Project,Bonus)

Eid	Ename	Salary
1	ABC	50,000
2	DEF	40,000
3	GHI	45,000

Fragment1

Eid	Bonus	Project
1	5000	P ₁
2	7000	P ₂
3	8000	P ₃

Fragment2

Reconstruction

Reconstruction of vertical fragmentation is performed by using **Full Outer Join** operation on fragments.

- select * from
fragment 1
NATURAL JOIN
fragment 2

Eid	Ename	Salary
1	ABC	50,000
2	DEF	40,000
3	GHI	45,000

Fragment1

Eid	Bonus	Project
1	5000	P ₁
2	7000	P ₂
3	8000	P ₃

Fragment2

Mixed (Hybrid) Fragmentation

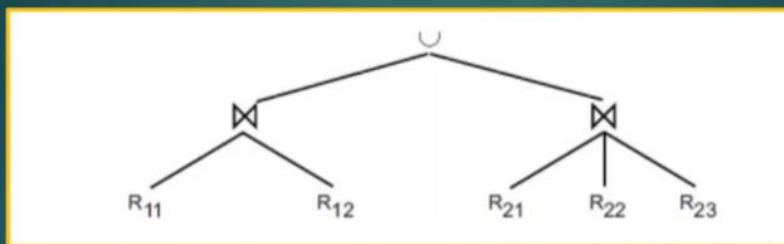
A combination of **horizontal and vertical fragmentation**.

- At first, generate a set of horizontal fragments; then generate vertical fragments from one or more of the horizontal fragments.
- At first, generate a set of vertical fragments; then generate horizontal fragments from one or more of the vertical fragments.

- Consider a relation: `Employee(EID, EName, EDept, ESalary, ELocation)`
- Step 1 – Horizontal Fragmentation:**
Fragment based on `ELocation`
 - `Emp_Delhi` : employees from Delhi
 - `Emp_Mumbai` : employees from Mumbai
- Step 2 – Vertical Fragmentation** (applied on `Emp_Delhi`):
 - `Emp_Delhi_1(EID, EName)`
 - `Emp_Delhi_2(EID, EDept, ESalary)`
- Final Result: Combination of horizontal and vertical fragments — a **hybrid** fragmentation strategy.

Reconstruction of Hybrid Fragmentation

39



The original relation in hybrid fragmentation is reconstructed by performing UNION and FULL OUTER JOIN.

1. Horizontal Fragmentation

Advantages:

- Queries accessing specific rows are faster.
- Easier to distribute data by location.

Disadvantages:

- Joins across fragments are expensive.
- May lead to data duplication if not disjoint.

2. Vertical Fragmentation

Advantages:

- Improves performance for queries accessing few columns.
- Reduces the size of tuples.

Disadvantages:

- Requires joins to access complete tuples.
- Increases complexity of query processing.

3. Hybrid Fragmentation

Advantages:

- Combines benefits of both horizontal and vertical fragmentation.
- Greater flexibility and optimization for complex access patterns.

Disadvantages:

- Most complex to implement and manage.
- Higher overhead for reconstruction and maintenance.

Data Allocation

- A database that consists of two or more data files allocated at different sites on a computer network is known as data location.
- The main objective of data allocation in distributed database is to be placed the data segment at different sites in such a way so that the total data transfer cost can be minimum while executing a set of queries.

Data Allocation Strategies

1. Centralized Allocation

- All data stored at a single site.
- Simple to manage but not fault-tolerant.
- Low data redundancy.

2. Partitioned Allocation

- Data divided into fragments and stored at different sites.
- Can be horizontal, vertical, or hybrid fragmentation.
- Reduces data access time for local users.

3. Replicated Allocation

- Copies of data (or fragments) are stored at multiple sites.
- Improves availability and fault tolerance.
- Increases update overhead and consistency complexity.

Factors Affecting Data Allocation

- Frequency of data access by different sites.
- Communication cost between sites.
- Query and transaction patterns.
- Site reliability and availability.
- Data security and privacy concerns.
- Network latency and bandwidth.

Information Requirements for Data Allocation

a. Database Information:

- Selectivity of fragments
- Size of a fragment

b. Application Information:

- Number of read accesses of a query to a fragment
- Number of update accesses of a query to a fragment
- A matrix indicating which queries update which fragments
- A similar matrix for retrievals
- Originating site of each query

c. Site Information:

- Unit cost of storing data at a site
- Unit cost of processing at a site

d. Network Information:

- Communication cost/frame between two sites
- Frame size (Frames are units of data in the Link Layer, e.g., WiFi, Bluetooth, Ethernet, etc.)

Allocation Model

- It has two components: A) Processing Component
B) Transmission Component

A) Processing Component:

Three factors :

1. Access cost(AC)
2. Integrity Enforcement(IE)
3. Concurrency Control cost(CC)

1. Access cost(AC)

$$AC_i = \sum_{\text{all sites}} \sum_{\text{all fragments}} (U_{ij} * UR_{ij} + r_{ij} * RR_{ij}) * LPc_k * X_{jk}$$

U_{ij} = equals 1 when update else 0 on f_j

UR_{ij} = no of update acesses

r_{ij} = equals 1 when read else 0 on f_j

RR_{ij} = no of read access

LPc_k = cost of access at a site K

X_{jk} = This ensures that f_j is stored at site K

The decision variable is x_{jk} , which is defined as $x_{jk} = \begin{cases} 1 & \text{if the fragment } f_j \text{ is stored at site } k \\ 0 & \text{otherwise} \end{cases}$

Integrity enforcement and concurrency control algorithm execution time costs are IE & CC respectively

B) Transmission Component

- It has two components-

1. Processing update cost
2. Processing retrieval cost

- In update queries it is necessary to inform all the sites where replicas exist,
- In addition, at the end of an update request, there is no data transmission back to the originating site other than a confirmation message,

1. Processing update cost

$$TCU_i = \sum_{\text{All sites}} \sum_{\text{All Fragments}} \text{update message cost} + \sum_{\text{All sites}} \sum_{\text{All Fragments}} \text{Acknowledge cost}$$

2. Processing Retrieval cost

- ✓ In retrieval queries, it is sufficient to access only one of the copies.
- ✓ The equation states that among all the sites with copies of the same fragment, only the site that yields the minimum total transmission cost should be selected for the execution of the operation.

$$TCR_i = \sum_{\text{All sites}} \min_{\text{all sites}} (\text{cost of retrieval command} + \text{cost of sending back the result})$$

Now the transmission cost function for query q_i can be specified as

Transmission cost function

$$TC_i = TCU_i + TCR_i$$